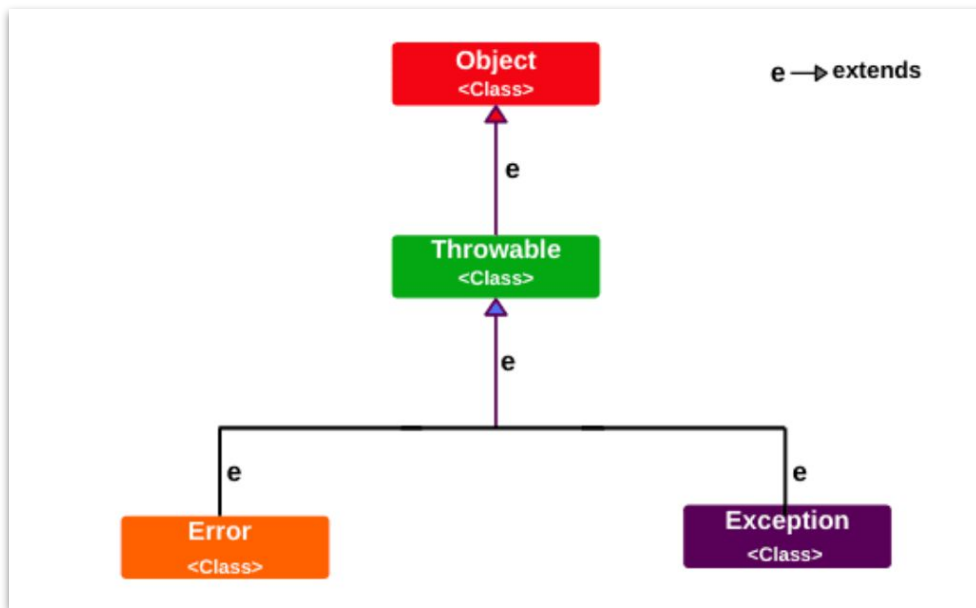


Jerarquía de excepciones

Esteban Álvarez

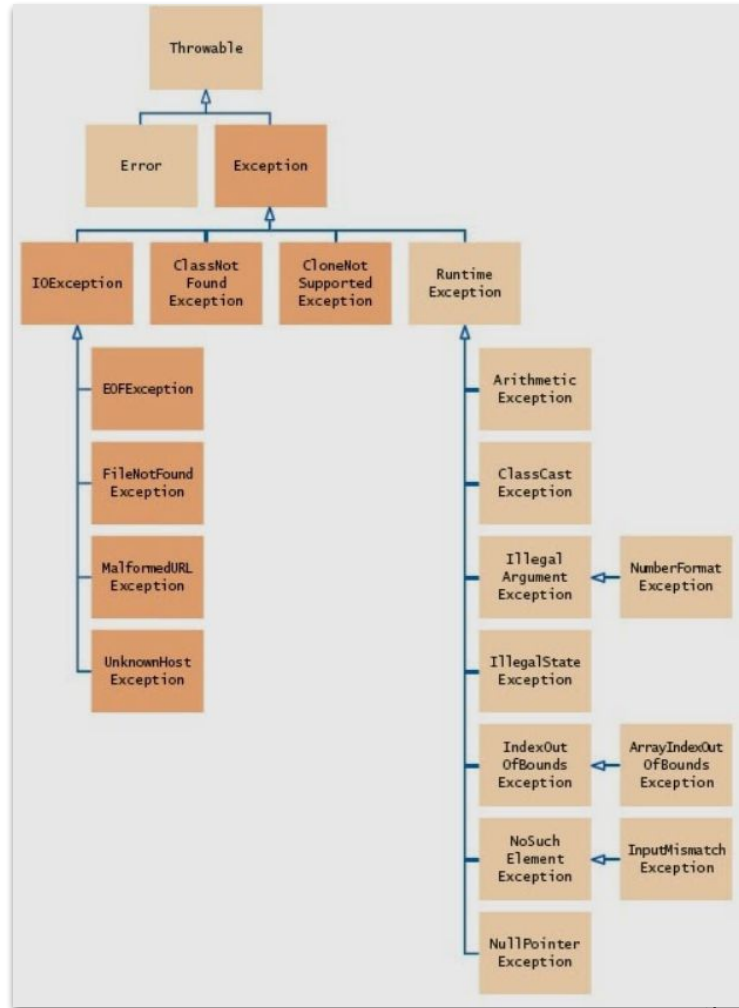
Jerarquía de excepciones

- En una aplicación podemos tener dos tipos de “**incidencias**”
 - Errores. Son errores en tiempo de compilación y **no se tratan**
 - Excepciones. Podemos/debemos manejar con bloques **try-catch**



Jerarquía de excepciones

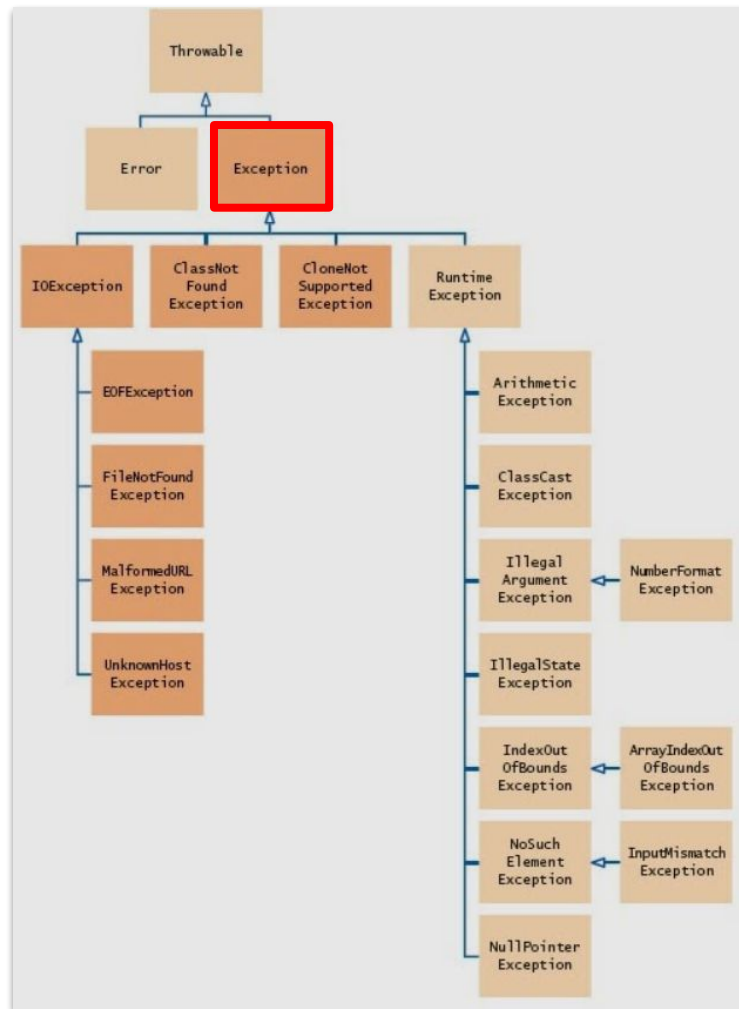
- Dentro de las excepciones hay **varios tipos**
- Entre ellas hay una relación de jerarquía
- El tratamiento de las excepciones de tipo **Runtime** es opcional



Tratamiento

- A la hora de tratar las excepciones hay que tener en cuenta **qué excepción capturamos dentro del catch**
- Cuando Eclipse autogenera el código del try-catch, **por defecto** aparece capturada `Exception`

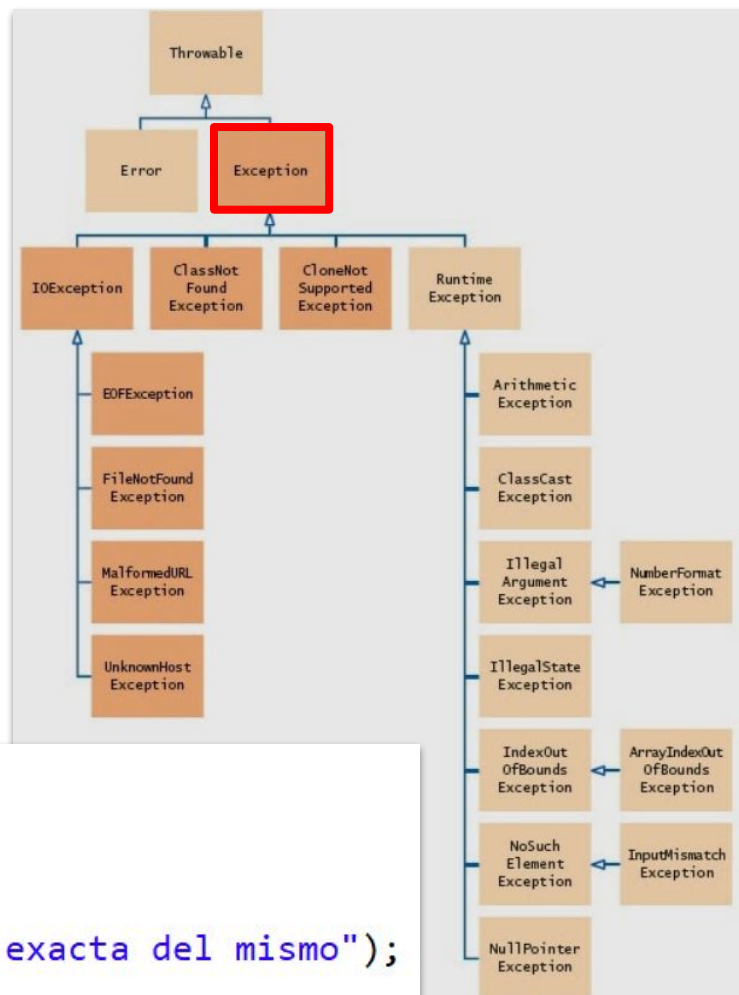
```
try {  
    int num=sc.nextInt();  
} catch (Exception e) {  
    // TODO Auto-generated catch block  
    e.printStackTrace();  
}
```



Tratamiento

- Al capturar `Exception` que está en la parte superior de la jerarquía **no podemos saber cuál es la causa exacta** de la excepción
- Puede ser de tipo `Runtime`, `ClassNotFoundException`, `FileNotFoundException`...

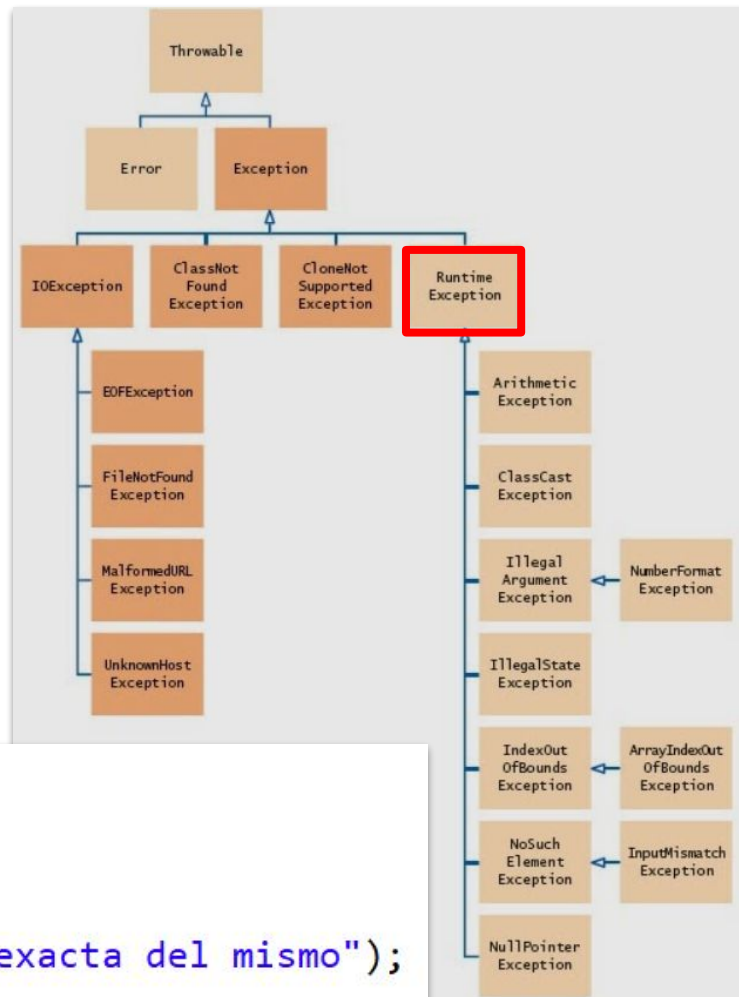
```
try {  
    int num=sc.nextInt();  
} catch (Exception e) {  
    System.out.println("Error");  
    System.out.println("No se puede saber la causa exacta del mismo");  
}
```



Tratamiento

- Pasa lo mismo al capturar
`RuntimeException` o `IOException`
- Ejemplo de Runtime: no se puede precisar si es de tipo Arithmetic, IndexOutOfBounds, NullPointerException...

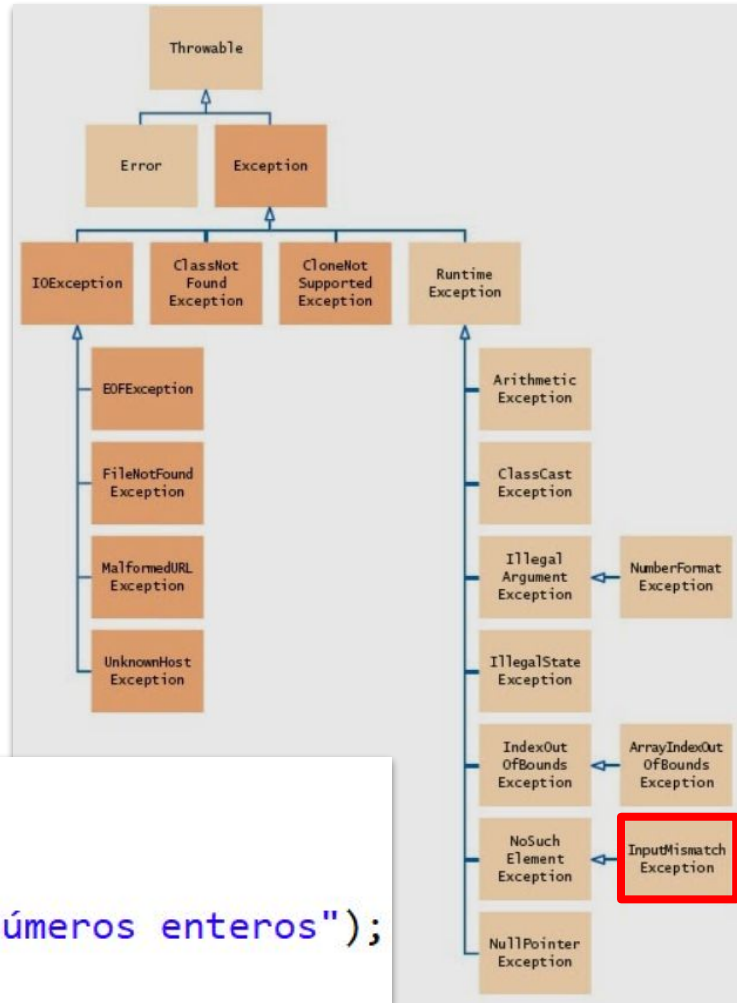
```
try {  
    int num=sc.nextInt();  
} catch (RuntimeException e) {  
    System.out.println("Error de tipo RunTime");  
    System.out.println("No se puede saber la causa exacta del mismo");  
}
```



Tratamiento

- La recomendación es capturar la **excepción más baja** dentro de la jerarquía
- Con eso se consigue dar un **mensaje preciso** sobre la causa de la excepción

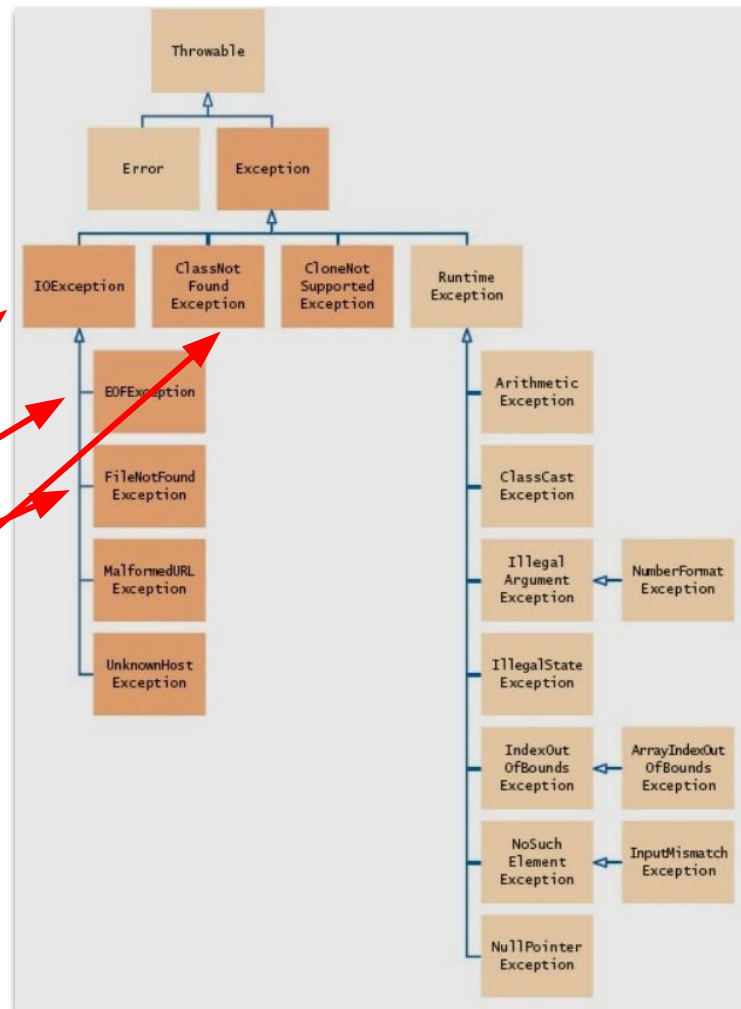
```
try {  
    int num=sc.nextInt();  
} catch (InputMismatchException e) {  
    System.out.println("Error. Introduce sólo números enteros");  
}
```



Múltiples catch

- Cuando el mismo bloque de código puede **generar distintos tipos de excepciones** añadimos varios catch al mismo try:

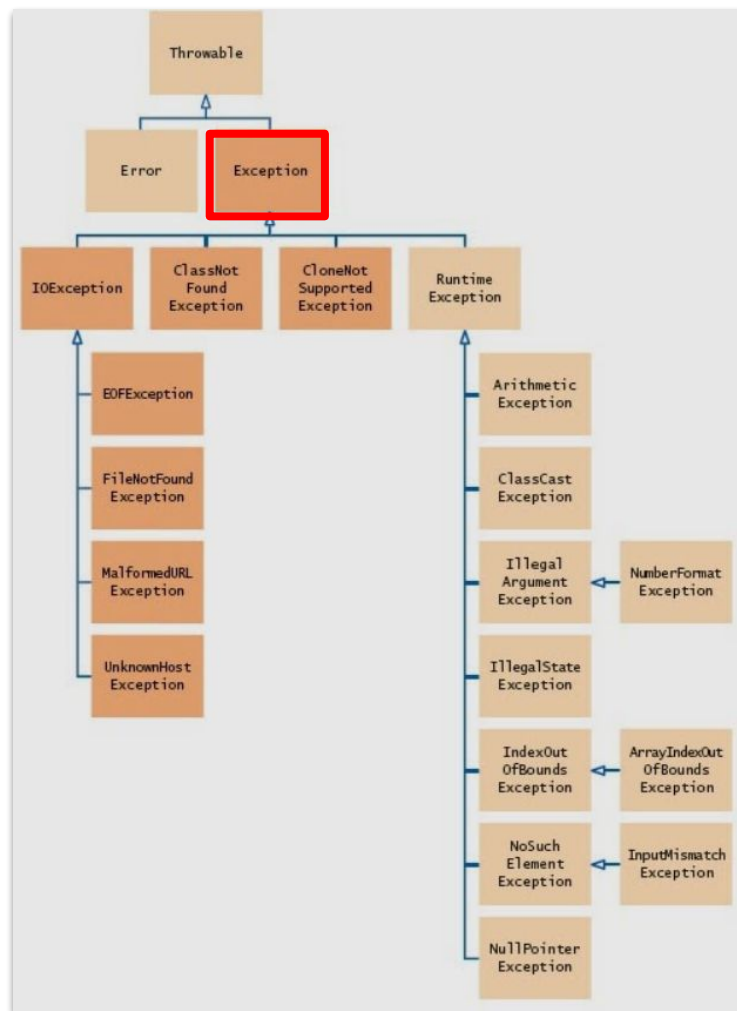
```
ObjectInputStream ois=null;
try {
    ois=new ObjectInputStream(new FileInputStream(fichero));
    Persona p;
    while((p=(Persona)ois.readObject())!=null) {
        System.out.println(p.toString());
    }
} catch (FileNotFoundException e) {
    System.out.println("No existe el fichero");
} catch (EOFException e) {
    // Fin del fichero, no se muestra ningún mensaje
} catch (IOException e) {
    System.out.println("Error al abrir el fichero");
} catch (ClassNotFoundException e) {
    System.out.println("Error al leer el fichero");
}
}
```



Tratamiento

- De nuevo se podría haber tratado sólo `Exception`
- Pero se repite el **problema**: No se puede especificar el error

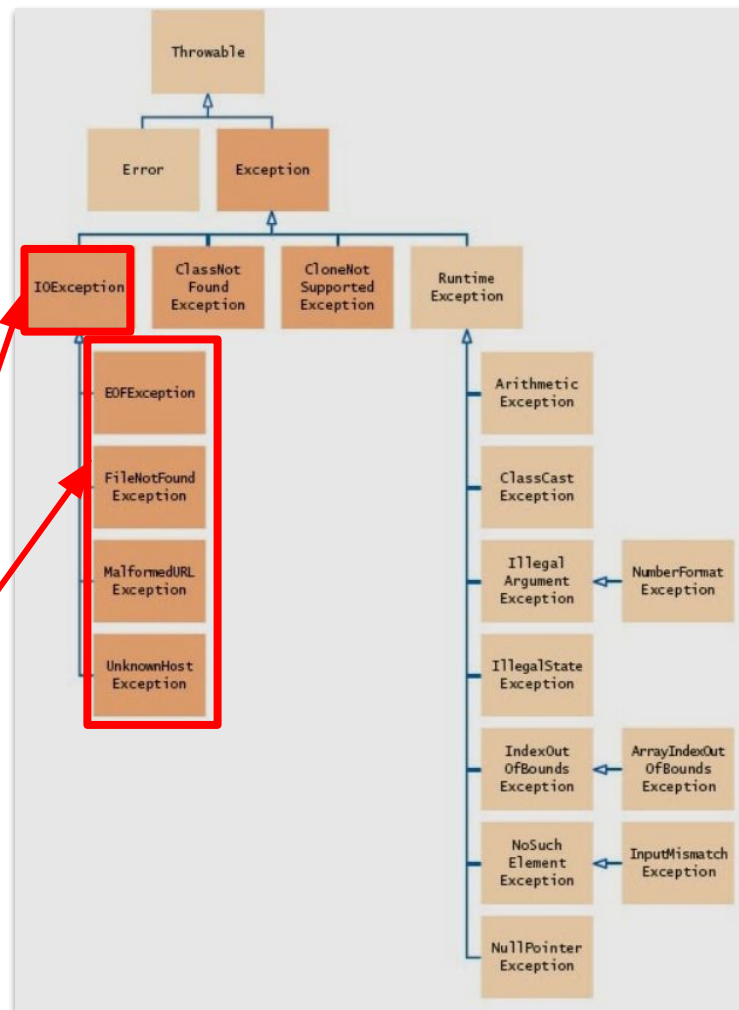
```
try {  
    ois=new ObjectInputStream(new FileInputStream(fichero));  
    Persona p;  
    while((p=(Persona)ois.readObject())!=null) {  
        System.out.println(p.toString());  
    }  
} catch (Exception e) {  
    System.out.println("Error al trabajar con el fichero");  
}
```



Múltiples catch. Orden

- Al tratar múltiples catch debemos hacerlo **en orden**
- **Primero** tratar las excepciones más bajas en la jerarquía
- Y **luego** las superiores

```
ObjectInputStream ois=null;
try {
    ois=new ObjectInputStream(new FileInputStream(fichero));
    Persona p;
    while((p=(Persona)ois.readObject())!=null) {
        System.out.println(p.toString());
    }
} catch (FileNotFoundException e) {
    System.out.println("No existe el fichero");
} catch (EOFException e) {
    // Fin del fichero, no se muestra ningún mensaje
} catch (IOException e) {
    System.out.println("Error al abrir el fichero");
} catch (ClassNotFoundException e) {
    System.out.println("Error al leer el fichero");
}
```



Múltiples catch. Orden

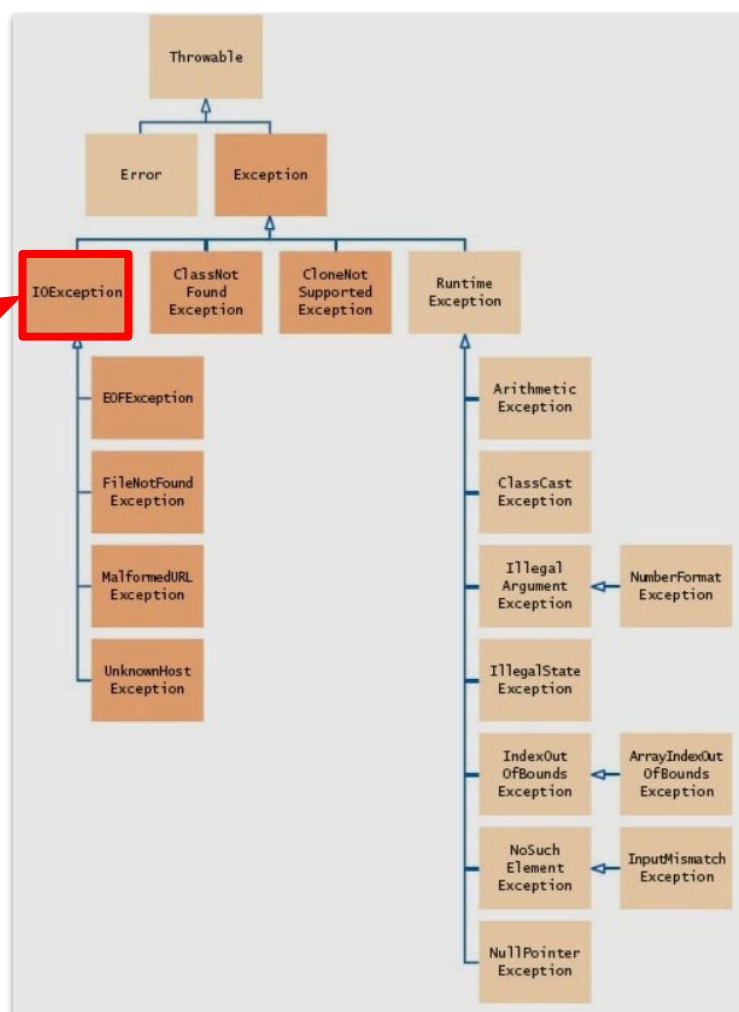
- Si se hace al revés, en el momento que se trata la excepción superior, el resto quedan sin tratar

```
} catch (IOException e) {  
    System.out.println("Error al abrir el fichero");  
} catch (FileNotFoundException e) {  
    System.out.println("No existe el fichero");  
} catch (EOFException e) {  
    //Si  
    System.out.println("Se ha alcanzado el final del fichero");  
} catch ( ) {  
    System.out.println("Otra excepción");  
}
```

Unreachable catch block for EOFException. It is already handled by the catch block for IOException

2 quick fixes available:

- Remove catch clause
- Replace catch clause with throws



Tipos de excepciones

- Busca en Internet información sobre los tipos de excepciones en Java y cuándo se produce cada una de ellas

Métodos y excepciones

- Eclipse nos informa de las excepciones que puede lanzar la ejecución de un método
- Ejemplo del método constructor `FileInputStream()`:

 `java.io.FileInputStream.FileInputStream(String name) throws FileNotFoundException`

Parameters:

name the system-dependent file name.

Throws:

[FileNotFoundException](#) - if the file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading.

[SecurityException](#) - if a security manager exists and its `checkRead` method denies read access to the file.

- Puede lanzar `FileNotFoundException` que es **obligatorio** capturar
- Puede lanzar `SecurityException` que, al ser de tipo `Runtime`, **no es obligatorio** capturar

Class `SecurityException`

```
java.lang.Object
  java.lang.Throwable
    java.lang.Exception
      java.lang.RuntimeException
        java.lang.SecurityException
```

Try-with-resources

- Para ampliar conocimientos busca información sobre la sentencia `try-with-resources` y cómo se puede aplicar al trabajo con streams

FIN!